

Weaker Language Models Need Colder Sampling, and Their Loss Says How Much

Anonymous authors

Paper under double-blind review

Draft. Every number in this version comes from a pilot run on WikiText-103 with a Qwen2.5-3B judge (Section 5). A replication on text written in 2026 with a stronger judge, plus a direct measurement of the self-consistent temperature, is in progress. Red text marks what changes when it finishes.

Abstract

Most people sample every language model at a temperature of 0.7 or 1.0. We ask what temperature a given model actually needs. We define T^* as the temperature at which a model’s samples are exactly as surprising to a stronger judge model as human-written text, and measure it for 14 base models from 70M to 2.8B parameters, including checkpoints from partway through training. T^* ranges from 0.62 to 0.83. It is set by the model’s loss, not its size: two models with the same loss and a $3.5\times$ difference in parameters have the same T^* , and a two-number formula fitted on Pythia alone predicts the three SmolLM2 models to within 0.014. This looks like it contradicts recent work showing that the temperature which makes a model *self-consistent* is the same at every scale. It doesn’t. We prove that any model whose log loss cannot be improved by rescaling its logits is self-consistent at temperature 1, however bad it is, so self-consistency cannot see model quality. Only a reference outside the model can. Against such a reference, the extra surprise of a model’s samples at temperature 1 is exactly the symmetric KL divergence between model and truth, roughly twice the model’s excess loss, and a first-order expansion turns that into a prediction for T^* that matches simulation. Every experiment ran on a single 16GB laptop.

1 Introduction

We finetuned a 297M-parameter model to chat and asked it the capital of Australia. At temperature 0.7 it said Melbourne. On another try it said Perth. At 0.7 it gave a sensible reply to “hi” 8 times out of 12; at 0.4 it managed 12 out of 12. Lowering the temperature didn’t make the model know more, but it stopped it from wandering into the parts of its distribution it had no business being in.

The usual advice is to pick a temperature between 0.7 and 1.0 and move on. That advice comes from large models. Nobody tells you what temperature a 70M or a 300M model wants, or what decides it. There is a large literature on *how* to sample: top- k (Fan et al., 2018), nucleus (Holtzman et al., 2020), typical (Meister et al., 2023), η (Hewitt et al., 2022) and min- p (Nguyen et al., 2024) sampling all reshape the distribution before drawing from it. Much less work asks how the right setting should change from one model to the next.

The closest recent answer points the other way. Cao et al. (2025) show that language models become *entropy-miscalibrated* as they generate (their per-step entropy drifts above their log loss) and that this barely improves with scale, so the temperature that fixes it is about the same from 0.5B to 72B parameters. If that were the whole story, one temperature would suit every model.

Our measurements say otherwise. We define the best temperature T^* as the one where a model’s samples are as surprising to a stronger judge model as real human text is, and measure it for 14 base models. It moves from 0.62 to 0.83, and it moves with loss (Figure 1).

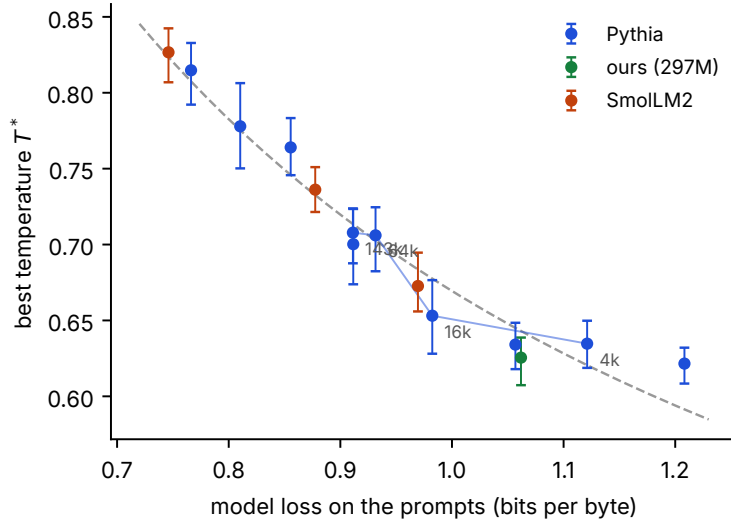


Figure 1: The best sampling temperature T^* against model loss for 14 base models. Error bars are 95% bootstrap intervals over prompts. The dashed curve is $T^* = 0.217 + 0.591 h/L$, where L is the model’s loss and h is the judge’s loss on human text. The thin line joins four checkpoints of Pythia-410M, labelled by training step. Lower loss means a hotter best temperature, for every family.

The two findings fit together once you separate two questions. “Is the model consistent with itself?” and “does the model match the world?” have different answers, and only the second one depends on how good the model is.

Contributions.

- **A measurement.** T^* for 14 base models from three families (Pythia, SmolLM2 and a 297M model we trained), 70M to 2.8B parameters, including Pythia checkpoints from partway through training (Section 6).
- **Loss, not size.** T^* is predicted by the model’s loss ($r^2 = 0.95$); adding parameter count to the fit changes nothing. Two models with equal loss and a $3.5\times$ size gap have equal T^* . A fit on Pythia alone predicts SmolLM2 to within 0.014.
- **Why self-consistency can’t see this.** A short proof that log-loss training makes any model self-consistent at temperature 1 regardless of quality, and an exact expression for how surprising its samples look to an outside reference: the symmetric KL divergence between model and truth (Section 4).
- **A first-order formula** for T^* in terms of the loss gap, checked against simulation.
- **Everything on a laptop.** All runs used one 16GB MacBook Air. Code, generations and results are released.

2 Background and related work

Temperature and truncation. Sampling at temperature T draws from $q_T(y) \propto q(y)^{1/T}$. Below 1 it sharpens the distribution, above 1 it flattens it. Truncation methods (Fan et al., 2018; Holtzman et al., 2020; Meister et al., 2023; Hewitt et al., 2022; Nguyen et al., 2024) cut off the low-probability tail instead of reweighting it. Hewitt et al. (2022) frame the tail as smoothing the model adds to avoid infinite loss, which

is close to how we think about temperature here. Finlayson et al. (2024) trace tail errors to the softmax bottleneck. We study plain temperature because it has one parameter, which makes “what value does this model need” a well-posed question.

Picking the temperature. Mirostat (Basu et al., 2021) adjusts truncation while decoding to hold perplexity at a target. That is the closest idea to ours, but it needs the target chosen by hand and does not say how the target should depend on the model. Li et al. (2025) find that larger models tolerate higher temperatures before benchmark scores drop, which agrees with our direction. Du et al. (2025) choose temperature for multi-sample reasoning, and Zhu et al. (2023) adapt it per token for code. None of these relate the temperature to the model’s loss.

Entropy calibration. Braverman et al. (2020) showed that the entropy of generated text drifts upward over long generations, and that a model’s entropy rate can be calibrated against its log loss. Cao et al. (2025) show that this miscalibration shrinks very slowly with scale, explain it with a power-law model of rare tokens, and find that the temperature giving zero calibration error is similar across model sizes. Our results agree with theirs. We argue they answer a different question from ours (Section 4).

Calibration and scaling. Temperature scaling is the standard fix for overconfident classifiers (Guo et al., 2017). Scaling laws relate loss to parameters and data (Kaplan et al., 2020; Hoffmann et al., 2022). Our result adds a decoding-side quantity that follows loss rather than parameters.

3 Three ways to define the right temperature

Let $p(\cdot | c)$ be the true next-token distribution after context c , and let the model have logits $\ell(c)$ and distribution $q(\cdot | c) = \text{softmax}(\ell(c))$. Sampling at temperature T uses $q_T = \text{softmax}(\ell/T)$. Write $H = \mathbb{E}_c H(p(\cdot | c))$ for the entropy of the truth and $L = \mathbb{E}_c \mathbb{E}_{y \sim p} [-\log q(y | c)]$ for the model’s loss. Three temperatures are worth naming.

Best for log loss, T_{cal} . The T that minimises $\mathbb{E}_c \mathbb{E}_{y \sim p} [-\log q_T(y | c)]$. This is classic temperature scaling (Guo et al., 2017).

Self-consistent, T_{self} . The T at which the model’s entropy equals its log loss, $\mathbb{E}_c H(q_T) = \mathbb{E}_c \mathbb{E}_{y \sim p} [-\log q_T(y | c)]$. This is entropy calibration (Braverman et al., 2020; Cao et al., 2025). A self-consistent model is exactly as surprised by its own samples as by real text.

Matches the world, T^* . The T at which the model’s samples are as surprising *to the truth* as real text is: $\mathbb{E}_c \mathbb{E}_{y \sim q_T} [-\log p(y | c)] = H$. In practice we cannot evaluate p , so we use a judge model J much stronger than the models being tested, and compare the judge’s bits per byte on samples against its bits per byte on human continuations of the same prompts.

T_{self} only involves the model. T^* needs something outside it. That difference turns out to be the whole story.

4 Why self-consistency can’t see model quality

We work one step at a time: contexts come from real text and we look at the next token. Section 4.1 discusses what changes over a whole generation.

Proposition 1 (Log-loss training gives self-consistency for free). *Let $q_\beta = \text{softmax}(\beta\ell)$ and suppose β^* minimises the log loss $\Lambda(\beta) = \mathbb{E}_c \mathbb{E}_{y \sim p} [-\log q_\beta(y | c)]$. Then at β^* the model’s average entropy equals its log loss:*

$$\mathbb{E}_c H(q_{\beta^*}) = \Lambda(\beta^*).$$

Proof. Write $Z_\beta(c) = \sum_y e^{\beta\ell_y(c)}$, so $-\log q_\beta(y | c) = -\beta\ell_y(c) + \log Z_\beta(c)$. Then

$$\Lambda(\beta) = \mathbb{E}_c [-\beta \mathbb{E}_p[\ell] + \log Z_\beta], \quad \mathbb{E}_c H(q_\beta) = \mathbb{E}_c [-\beta \mathbb{E}_{q_\beta}[\ell] + \log Z_\beta].$$

Since $\frac{d}{d\beta} \log Z_\beta = \mathbb{E}_{q_\beta}[\ell]$, we get $\Lambda'(\beta) = \mathbb{E}_c[\mathbb{E}_{q_\beta}[\ell] - \mathbb{E}_p[\ell]]$. Λ is convex in β , so at the minimum $\mathbb{E}_c \mathbb{E}_{q_{\beta^*}}[\ell] = \mathbb{E}_c \mathbb{E}_p[\ell]$. Subtracting the two lines above at β^* gives $\mathbb{E}_c H(q_{\beta^*}) - \Lambda(\beta^*) = -\beta^* (\mathbb{E}_c \mathbb{E}_{q_{\beta^*}}[\ell] - \mathbb{E}_c \mathbb{E}_p[\ell]) = 0$. $\square$

A network trained with log loss can rescale its own logits through its final layer, so in practice $\beta^* \approx 1$, and the proposition says $T_{\text{self}} = T_{\text{cal}} = 1$ for one step. Nothing in the proof depends on how close q is to p . A model that is badly wrong is still self-consistent, because it is wrong about the world in the same way it is wrong about its own samples.

Now look at the same model from outside.

Corollary 1 (What an outside reference sees). *Under the conditions of Proposition 1 with $\beta^* = 1$, the extra surprise of the model’s samples to the truth, at temperature 1, is the symmetric KL divergence between truth and model:*

$$\mathbb{E}_c \mathbb{E}_{y \sim q}[-\log p(y | c)] - H = \mathbb{E}_c [\text{KL}(p \| q) + \text{KL}(q \| p)].$$

Proof. $\mathbb{E}_q[-\log p] = H(q) + \text{KL}(q \| p)$. By Proposition 1, $\mathbb{E}_c H(q) = L = H + \mathbb{E}_c \text{KL}(p \| q)$. Substitute. $\square$

So unless the model is perfect, its samples at $T = 1$ are too surprising, and $T^* < 1$. The excess is at least the model’s excess loss $L - H = \mathbb{E}_c \text{KL}(p \| q)$, and for small errors the two KL terms are about equal, so the excess is roughly twice the loss gap. Lowering T reduces it. Expanding the judged surprise to first order around $T = 1$, using $\frac{d}{dT} \mathbb{E}_{q_T}[f] \big|_{T=1} = \text{Cov}_q(\log q, -f)$, gives

$$T^* \approx 1 - \frac{\mathbb{E}_c [\text{KL}(p \| q) + \text{KL}(q \| p)]}{\mathbb{E}_c \text{Cov}_q(\log q, \log p)} \approx 1 - \frac{2(L - H)}{C}, \quad (1)$$

where C measures how spread out the log-probabilities are (for a good model, close to the variance of $\log p$, sometimes called varentropy). Worse loss, lower T^* , at a rate set by how varied the text is.

A toy model. We checked this with a simulation (Figure 2). The truth is 1,500 Zipf-shaped next-token distributions over 4,000 tokens with random steepness. A model of quality σ has logits $\log p + \sigma \varepsilon$ with Gaussian noise ε , rescaled by the log-loss-optimal temperature, which is what training would do. As σ grows, T_{cal} and T_{self} stay at exactly 1.000, as Proposition 1 says they must. T^* falls from 1.0 to 0.47. The excess surprise at $T = 1$ matches the symmetric KL to four decimal places, and Equation 1 predicts T^* to within 0.003 for loss gaps up to 7% of H and within 0.02 at 17%. Plotted against H/L , T^* is close to a straight line ($r^2 = 0.98$), the same form that fits our real models best (Section 6).

4.1 Over a whole generation

Real generations feed the model its own samples as context, and errors pile up: this is the entropy drift of Braverman et al. (2020) and Cao et al. (2025). It pushes every model’s measured T_{self} below 1, by an amount that barely depends on scale. It also pushes T^* below the one-step value. Together, the theory predicts:

1. **P1.** $T_{\text{cal}} \approx 1$ for every model.
2. **P2.** T_{self} below 1 and roughly flat across models, as in Cao et al. (2025).
3. **P3.** T^* below T_{self} for weak models, rising toward it as loss falls.
4. **P4.** T^* measured on the second half of a continuation lower than on the first half, by a similar amount for every model.
5. **P5.** Text sampled at T^* preferred over text at the default temperatures.

[P1, P2 and P4 are measured directly in the full run (the script now records the model’s loss at 25 temperatures, its surprise at its own samples, and judge scores for each half of the continuation). P5 is the preference study in Section 8.]

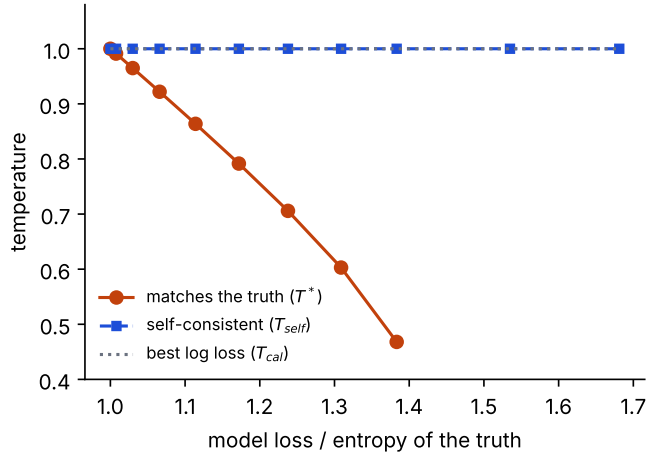


Figure 2: Toy model. Models get worse to the right. The log-loss-optimal temperature and the self-consistent temperature stay at 1 for every model (Proposition 1). The temperature that matches the truth falls steadily as the model gets worse.

5 Setup

Text. 200 passages from WikiText-103 (Merity et al., 2016). Each gives a 40-word prompt and the next 80 words as the human continuation. [Full run: 300+ passages per model from English Wikipedia articles created in 2026, after every model’s training cutoff.]

Models. Pythia 70M, 160M, 410M, 1B, 1.4B and 2.8B (Biderman et al., 2023); Pythia-410M at training steps 4k, 16k and 64k and Pythia-1.4B at step 16k; SmolLM2 135M, 360M and 1.7B (Ben Allal et al., 2025); and a 297M Llama-style model we pretrained on 6.1B tokens. Pythia runs in fp32 (fp16 for 2.8B) because it breaks in bf16: on a test paragraph Pythia’s loss went from 4.15 to 5.90. [Full run adds Pythia-160M at seven checkpoints from step 1k to 64k and Pythia-1B at 4k and 16k.]

Sampling. Pure temperature sampling, no top- k or top- p , 160 new tokens truncated to 80 words, temperatures 0.1 to 1.5 in steps of 0.1, fixed seeds. Each model generates for each prompt at every temperature.

Judge. Qwen2.5-3B (Qwen Team, 2024) scores every continuation in bits per byte given its prompt, so models with different tokenizers are comparable. It also scores the human continuations, giving the human level $h = 0.766$ bits per byte. [Full run: Qwen2.5-7B, 4-bit, in MLX.]

Measuring T^* and L . For each model, the judge’s bits per byte rises with temperature (Figure 3); T^* is where the curve crosses h , interpolated linearly between grid points. We bootstrap prompts 300 times for 95% intervals. L is the model’s own bits per byte on the human continuations.

Compute. One MacBook Air (M5, 16GB, no fan). The pilot ran overnight for 9.6 hours. Generation held about 345 tokens per second for Pythia-410M for the whole run, so the fanless chassis did not throttle noticeably.

6 Results

6.1 Every model wants a temperature below 1

Figure 3 shows the judge’s bits per byte on each model’s samples as temperature rises. At $T = 1$ every model is too surprising: from 1.05 bits per byte for SmolLM2-1.7B and Pythia-2.8B up to 1.67 for Pythia-70M,

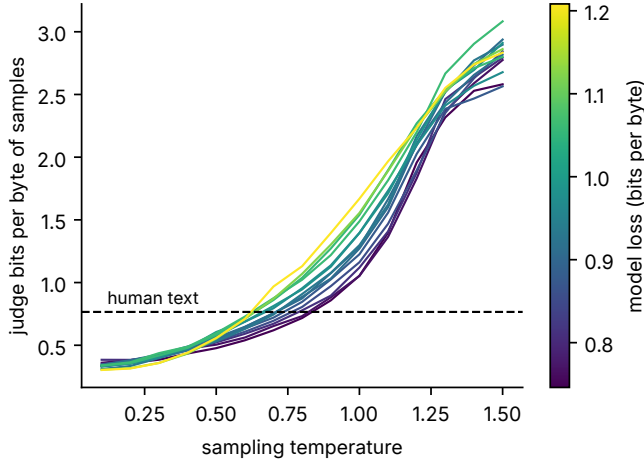


Figure 3: How surprising each model’s samples are to the judge, by temperature. Colour is the model’s loss. The dashed line is human text. Each model’s T^* is where its curve crosses the line, and worse models cross earlier.

model	params	loss L	T^*	95% interval	prompts
SmolLM2-1.7B	1711M	0.746	0.827	0.807–0.842	100
Pythia-2.8B	2775M	0.766	0.815	0.792–0.833	100
Pythia-1.4B	1415M	0.810	0.778	0.750–0.806	100
Pythia-1B	1012M	0.856	0.764	0.746–0.783	200
SmolLM2-360M	362M	0.878	0.736	0.721–0.751	200
Pythia-410M	405M	0.911	0.708	0.688–0.724	200
Pythia-1.4B @ 16k	1415M	0.912	0.700	0.674–0.723	100
Pythia-410M @ 64k	405M	0.932	0.706	0.682–0.725	100
SmolLM2-135M	135M	0.970	0.673	0.656–0.695	200
Pythia-410M @ 16k	405M	0.982	0.653	0.628–0.677	100
Pythia-160M	162M	1.057	0.634	0.618–0.648	200
ours-297M	297M	1.062	0.626	0.607–0.639	200
Pythia-410M @ 4k	405M	1.121	0.635	0.619–0.650	200
Pythia-70M	70M	1.208	0.622	0.608–0.632	200

Table 1: All 14 models, sorted by loss (bits per byte on the human continuations). T^* is the judge-matched temperature. The judge’s loss on human text is $h = 0.766$.

against 0.77 for human text. Every curve crosses the human line between 0.6 and 0.85. At the common default of 0.7, the six weakest models are too hot, the five best are too cold, and only the three in the middle have intervals that include 0.7.

6.2 Loss predicts T^*

Table 1 lists all 14 models. T^* rises steadily as loss falls. Table 2 compares ways of predicting it. The best is linear in h/L :

$$T^* \approx 0.217 + 0.591 \frac{h}{L}, \quad (2)$$

with $r^2 = 0.955$ and a leave-one-out error of 0.014 on average (0.042 at worst). A straight line in L does worse ($r^2 = 0.896$) because the relationship bends at both ends; the h/L form is the one the toy model produces. Parameter count alone gets $r^2 = 0.68$.

predictor	r^2	leave-one-out error	worst
h/L	0.955	0.014	0.042
$\log L$	0.931	0.018	0.058
L	0.896	0.023	0.075
L and $\log(\text{params})$	0.899		
$\log(\text{params})$	0.681	0.038	0.075

Table 2: Linear predictors of T^* across 14 models. Adding parameters to loss raises r^2 by 0.003; its coefficient is 0.017 per factor of 10.

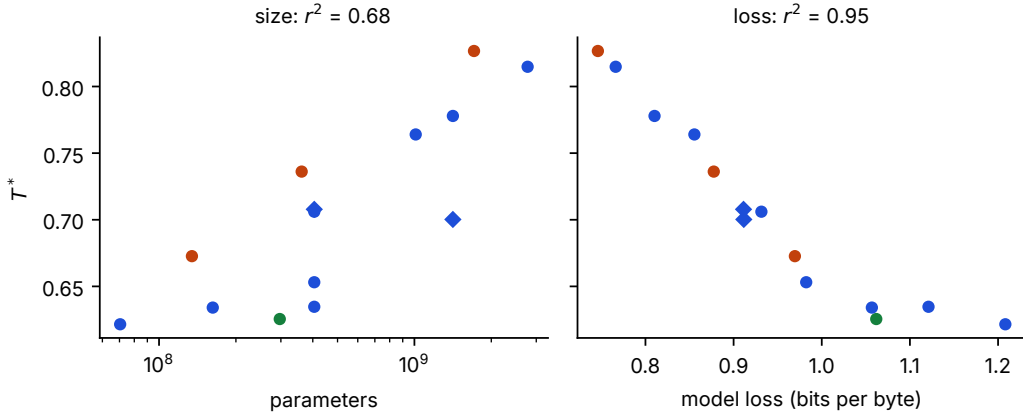


Figure 4: T^* against parameters (left) and against loss (right). Diamonds are the matched pair Pythia-410M and Pythia-1.4B at step 16k: same loss, $3.5\times$ different size, same T^* .

6.3 Size adds nothing once you know the loss

Pythia-1.4B at step 16k and the finished Pythia-410M have the same loss (0.912 and 0.911) and $3.5\times$ different parameter counts. Their T^* values are 0.700 and 0.708, inside each other’s intervals. SmolLM2-135M and Pythia-410M at step 16k make a second pair: similar loss (0.970 and 0.982), $3\times$ apart in size, T^* of 0.673 and 0.653. Regressing T^* on loss and $\log(\text{params})$ together gives the parameter term a coefficient of 0.017 per factor of ten and improves r^2 from 0.896 to 0.899. Figure 4 shows both views.

6.4 Inside one training run

The cleanest test holds the architecture fixed and lets only training change. Pythia-410M at steps 4k, 16k, 64k and 143k (the final model) has loss 1.121, 0.982, 0.932 and 0.911, and T^* of 0.635, 0.653, 0.706 and 0.708. The same network wants a hotter temperature as it learns, and it moves along the same curve as the other models (Figure 1). [Full run: seven Pythia-160M checkpoints and two Pythia-1B checkpoints.]

6.5 Across model families

We fitted Equation 2 on the 10 Pythia models only and used it to predict the others (Table 3). It gets the three SmolLM2 models to within 0.014, all inside their bootstrap intervals. Our 297M model comes out 0.022 colder than predicted, just outside its interval. That model was trained on a different data mix with a different tokenizer, and we do not yet know whether the gap is real.

model	predicted	measured	error	inside interval
SmolLM2-1.7B	0.821	0.827	+0.006	yes
SmolLM2-360M	0.733	0.736	+0.003	yes
SmolLM2-135M	0.686	0.673	−0.014	yes
ours-297M	0.647	0.626	−0.022	no

Table 3: Fitted on Pythia only ($T^* = 0.238 + 0.567 h/L$), tested on the other families.

6.6 A second check that didn’t work

We also looked for the temperature where the share of distinct words in a sample matches human text. It came out between 0.90 and 0.93 for every model, with no trend. Eighty words is too short for distinct-word counts to separate models, so we do not use this measure further.

7 Using it

Equation 2 needs two numbers: your model’s loss on some text it has not seen, and a strong model’s loss on the same text. In our setting the rule of thumb is simple. Models around 1.2 bits per byte want about 0.62. Models around 0.9 want about 0.71. Models near the judge’s own level want about 0.82. The usual 0.7 is close for mid-sized models and wrong at both ends: too hot for the small models people run on phones and laptops, and too cold for the best models we tested. The exact constants depend on the text domain and the judge, which is why the full run repeats everything with a second judge.

8 Limitations

- **The judge is not much stronger than the best models.** SmolLM2-1.7B (0.746) beats Qwen2.5-3B on these prompts (0.766), and Pythia-2.8B ties it. The method assumes the judge knows real text better than the models it judges, so the top of the curve is the least reliable part. [Full run uses Qwen2.5-7B.]
- **Contamination.** WikiText-103 is old Wikipedia, which appears in the Pile and so in Pythia’s training data. That can make Pythia’s loss look better than it is. [Full run uses 2026 articles.] Fresh Wikipedia has its own problem: some new articles may have been drafted with language models, so “human text” is not guaranteed.
- **“Best” is defined by the judge.** Matching the judge’s surprise at human text is one definition of the right temperature. [Planned: a pairwise preference test (an instruction-tuned judge picks the more natural of two continuations, order randomised, T^* against 0.7 and 1.0), plus 50 blind human pairs.]
- **Scope.** One domain (encyclopedia text), base models only, at most 2.8B parameters, 80-word continuations, plain temperature with no truncation. Chat models are tuned differently and may behave differently.
- **Few prompts for the largest models.** The four largest models have 100 prompts each, so their intervals are the widest.
- **The theory is one-step.** Propositions 1 and 1 describe next-token prediction from real contexts. Error accumulation over a generation is handled by citing Cao et al. (2025), not derived.

9 Conclusion

The temperature that makes a model consistent with itself is the same for every model, and we proved it has to be: log-loss training guarantees it. The temperature that makes a model’s text look like human text

is different for every model, and it is set by how good the model is. Measure your model’s loss on fresh text, and Equation 2 tells you roughly how cold to sample. Small models need it most.

Broader impact statement

This work helps small models that run on personal devices produce more coherent text with no extra compute. We do not see a meaningful route to harm from choosing a sampling temperature.

References

- Sourya Basu, Govardana Sachitanandam Ramachandran, Nitish Shirish Keskar, and Lav R. Varshney. Mirostat: A neural text decoding algorithm that directly controls perplexity. In *International Conference on Learning Representations*, 2021. arXiv:2007.14966.
- Loubna Ben Allal et al. Smolm2: When smol goes big. *arXiv preprint arXiv:2502.02737*, 2025.
- Stella Biderman et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, 2023. arXiv:2304.01373.
- Mark Braverman, Xinyi Chen, Sham Kakade, Karthik Narasimhan, Cyril Zhang, and Yi Zhang. Calibration, entropy rates, and memory in language models. In *International Conference on Machine Learning*, 2020. arXiv:1906.05664.
- Steven Cao, Gregory Valiant, and Percy Liang. On the entropy calibration of language models. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2511.11966.
- Weihua Du, Yiming Yang, and Sean Welleck. Optimizing temperature for language models with multi-sample inference. In *International Conference on Machine Learning*, 2025. arXiv:2502.05234.
- Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In *Proceedings of ACL*, 2018. arXiv:1805.04833.
- Matthew Finlayson, John Hewitt, Alexander Koller, Swabha Swayamdipta, and Ashish Sabharwal. Closing the curious case of neural text degeneration. In *International Conference on Learning Representations*, 2024. arXiv:2310.01693.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning*, 2017. arXiv:1706.04599.
- John Hewitt, Christopher D. Manning, and Percy Liang. Truncation sampling as language model desmoothing. In *Findings of EMNLP*, 2022. arXiv:2210.15191.
- Jordan Hoffmann et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020. arXiv:1904.09751.
- Jared Kaplan et al. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Lujun Li, Lama Sleem, Niccolò Gentile, Geoffrey Nichil, and Radu State. Exploring the impact of temperature on large language models: Hot or cold? *arXiv preprint arXiv:2506.07295*, 2025.
- Clara Meister, Tiago Pimentel, Gian Wiher, and Ryan Cotterell. Locally typical sampling. *Transactions of the Association for Computational Linguistics*, 2023. arXiv:2202.00666.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

Minh Nguyen et al. Turning up the heat: Min-p sampling for creative and coherent llm outputs. *arXiv preprint arXiv:2407.01082*, 2024.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Yuqi Zhu et al. Hot or cold? adaptive temperature sampling for code generation with large language models. *arXiv preprint arXiv:2309.02772*, 2023.

A Proof of the first-order formula

Let $g(T) = \mathbb{E}_c \mathbb{E}_{y \sim q_T} [-\log p(y \mid c)]$. With $q_T \propto q^{1/T}$, $\frac{d}{dT} \mathbb{E}_{q_T}[f] = -\frac{1}{T^2} \text{Cov}_{q_T}(\log q, f)$, so $g'(1) = \mathbb{E}_c \text{Cov}_q(\log q, \log p)$, which is positive whenever model and truth broadly agree on which tokens are likely. Corollary 1 gives $g(1) - H = \mathbb{E}_c[\text{KL}(p \parallel q) + \text{KL}(q \parallel p)]$. Solving $g(1) + (T^* - 1)g'(1) = H$ gives Equation 1. Since both KL terms equal $\frac{1}{2}\mathbb{E}\delta^2$ to second order in the log-probability error δ , their sum is about $2(L - H)$.

In the simulation the predicted and true T^* were 0.991 and 0.991 at a 0.8% loss gap, 0.965 and 0.965 at 3%, 0.925 and 0.922 at 7%, 0.872 and 0.864 at 11%, and 0.811 and 0.792 at 17%.

B Simulation details

Vocabulary 4,000, 1,500 contexts. Context i has $p_i(k) \propto k^{-\alpha_i}$ over a ranking of tokens, with α_i uniform on $[0.9, 2.2]$, giving entropies from near zero to several nats ($H = 3.24$ nats on average). Model logits are $b(\log p + \sigma\varepsilon)$ with ε standard Gaussian per token and context and b chosen to minimise log loss. T^* , T_{self} and T_{cal} are found by root finding and bounded minimisation. Code is in the supplement.

C Pythia in bfloat16

Pythia was trained in fp16. Loading it in bf16 raised its loss on a test paragraph from 4.15 to 5.90. Every Pythia model here runs in fp32, except Pythia-2.8B in fp16 to fit in 16GB. The judge runs in bf16.

D Full curves

[Per-model judge curves with intervals, and the human-level line, for every model in the full run.]

E Reproducing

[One command to rebuild the prompts, one to run every model (it resumes where it stopped), one to make every table and figure. Hours per model on the laptop.]